NFFH architecture exploration: conversation export

Course: Software Engineering using AI · System: Not Far(m) From Home · 29 October 2026

STUDENT

Attachment: NFFH.txt (reproduced in the appendix)

ROLE You are an experienced senior software engineer and architect.

SCENARIO University assignment for the course "Software Engineering using AI". I'm a student of applied informatics. The goal is for ME to understand the architectures.

STANDING RULE (whole session) Keep a notes file called "notes.txt". Create it now, then update it after EVERY one of your replies for the rest of this conversation, without waiting for me to ask. Append the important bits from each exchange: what was proposed, key trade-offs, my questions and your answers, and open issues. It doesn't need to be a 1:1 record. Keep it organised by architecture and topic rather than as a raw chat log. At the end of each reply, add one line confirming what you added to notes.txt.

OBJECTIVE Help me explore 3 meaningfully different architectures for the system described in the attached file "NFFH.txt".

STEPS FOR THIS PROMPT

1. Read the attached "NFFH.txt".
2. Create "notes.txt".
3. Propose 3 architectures with only a short overview each (2–3 sentences + a simple diagram).
4. Update notes.txt.
5. Stop there. We will then go through each architecture one at a time: benefits, trade-offs, difficult parts, and painful low-level implementation details.

CLAUDE

I'll set up the notes file, then lay out the three options.

Here are three meaningfully different ways to build NFFH. They differ mainly in **how the backend is split up and who runs it**.

A. Layered Modular Monolith

One backend application, deployed as a single unit, organised internally into modules (Users/Auth, Catalog, Areas, Orders) with classic layers (controller → service → repository). One relational database holds everything, and a SPA or server-rendered frontend talks to it via a REST API. This is the simplest to build, test and deploy, and it fits a system of this size naturally.

```
[Browser: Client / Farmer / Admin UI]
                |  HTTPS / REST (JSON)
                v
```

```
+----------------------------------------+
|        NFFH Backend (single app)       |
|  Controllers (REST endpoints)          |
|  --------------------------------------|
|  Auth | Catalog | Areas | Cart/Orders  |  <- modules
|  --------------------------------------|
|  Repositories (ORM / SQL)              |
+----------------------------------------+
                |
                v
       [ Relational DB (PostgreSQL) ]
```

B. Microservices with an API Gateway

The system is split into independently deployable services along business boundaries (Identity, Catalog/Inventory, Orders, Geo/Areas). Each service owns its own database. A gateway routes requests and validates tokens. Services communicate via REST and asynchronous events over a message broker (e.g. "OrderPlaced" → reserve stock). Each part scales and evolves independently, but you pay for it in distributed-systems complexity.

```
            [Browser UI]
                 |
                 v
          [ API Gateway ]  (routing, JWT check)
        /      |       |        \
       v       v       v         v
[Identity] [Catalog] [Orders] [Geo/Areas]
    |          |        |          |
  [DB]       [DB]     [DB]       [DB]
      \        |        |         /
       +---[ Message Broker ]----+
            (events: OrderPlaced,
             ProductUpdated, ...)
```

C. Serverless / Backend-as-a-Service (BaaS)

There is no backend server that you run yourself. The frontend talks directly to a managed platform (e.g. Supabase or Firebase) that provides authentication, a hosted database with **row-level security rules** ("a farmer can only edit their own products"), and file storage. The few operations that need trusted logic, like completing an order and decrementing stock atomically, run as small cloud functions. You write very little backend code. The catch is that authorisation moves into database rules, and you become tied to the vendor.

```
           [Browser UI (SPA)]
          /        |         \
         v         v          v
  [Managed    [Managed DB +   [Cloud Functions]
   Auth]       Row-Level       - completeOrder()
               Security]       - admin ops
```

```
                    ^                |
                    +----------------+
          (all hosted by the BaaS provider)

    Google Maps: plain link built from farm lat/long
```

When you're ready, pick one to go deep on first: benefits, trade-offs, hard parts, and the painful low-level details.

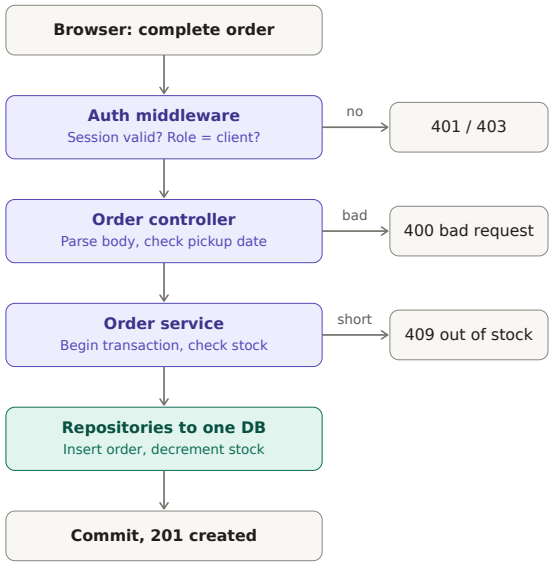
File sent: notes.txt

📝 notes.txt: created it with a system summary of NFFH and the three architecture overviews with mini-diagrams, plus an "Open issues" list. The open issues cover order and stock semantics, how "hot products" is defined, how areas work, multi-farm carts, and session strategy.

STUDENT

lets begin with the first one, alongside a generated flowchart at the beginning to help me visualize whats going on

CLAUDE



Flowchart: "Complete order" request in Architecture A (layered modular monolith)

The flowchart traces one request, "complete order" (story 12), through the monolith. It's the most interesting request because it touches auth, validation, business rules and the database all at once. Every other feature follows the same top-to-bottom path. You can click any box to dig into it.

Architecture A: Layered Modular Monolith

How it's organised

There is one codebase, one running process and one database. Inside the code you cut the app two ways at once:

- **Vertically into modules**, one per business area. Each module owns its own tables:
  - `auth/users`: stories 1–5, 17–24, 25, 27–29
  - `catalog`: stories 6, 8, 14–16, 33–38
  - `areas`: stories 7, 13, 30–32, 35, 39
  - `orders`: stories 9–12, 26, 40, 41
- **Horizontally into layers** inside each module:
  - **controller**: HTTP, JSON, input validation
  - **service**: business rules, transactions
  - **repository**: SQL/ORM

A typical stack would be Spring Boot, Django, NestJS or ASP.NET with PostgreSQL. The frontend can be a SPA (React/Vue) talking REST, or server-rendered pages.

The golden rule is that **modules talk to each other only through service interfaces**. For example, `orders` calls `catalog.reserveStock()`; it never queries the `products` table directly. That one rule is what makes it a *modular* monolith and not a "big ball of mud".

Benefits

- **One ACID transaction covers everything.** Creating the order and decrementing stock happen atomically, in one `BEGIN … COMMIT`. In Architecture B, this same operation becomes a hard distributed problem.
- **Simple to run.** You deploy one artifact plus one DB. Hosting is cheap, and local dev is one `docker compose up`.
- **Easy debugging and testing.** A single stack trace shows the whole request. Integration tests hit a real DB without needing mocks of other services.
- **Fast to build.** Refactoring across modules is just an IDE rename. This is ideal for a student team and a small platform.
- **Right-sized.** A regional farm marketplace has low, predictable traffic, so one well-indexed Postgres can handle it easily.

Trade-offs

- **It scales as one unit.** If catalog browsing gets heavy, you replicate the whole app. In practice, running a few copies behind a load balancer is fine at this scale.
- **Shared fate.** A memory leak in admin reports can take down checkout too.
- **Every change redeploys everything.** One stack and one release cycle.
- **Boundaries are only enforced by discipline.** Nothing physically stops a developer from joining `orders` to `users` in SQL. Over time, coupling creeps in unless you enforce module rules, for example with ArchUnit or lint rules on imports.
- **Team scaling.** Many developers in one repo means merge conflicts. That isn't a real issue for your project.

Difficult parts (design-level)

1. **Authorisation is more than authentication.** There are three roles, and the hard part is ownership checks: a farmer may only modify *their own* products, and a client may only see *their own* orders. If you forget a check, you get the classic IDOR bug: `PUT /products/42` works for any logged-in farmer. Admin stories 36–38 duplicate farmer actions, so reuse the same service methods with a different policy rather than writing a second code path.
2. **A cart with products from several farms.** Since pickup happens *at a farm*, one checkout probably has to become **one order per farm**, each with its own pickup date. This decision changes your data model, so make it early.
3. **Deleting things that other things reference.** These cases need policies, not cascades:
   - An admin deletes an area that has farms in it (story 39).
   - A farmer removes a product that's in someone's order (story 15).
   - An admin deletes a malicious user who has open orders (story 25).

   The usual answer is soft deletes (`deleted_at`) plus foreign keys with `RESTRICT`.

Painful low-level details

- **Race on the last kilo.** Two clients check out the last 2 kg of tomatoes at the same moment. Both read `stock = 2`, both succeed, and you've oversold. There are three standard fixes:
  - an atomic conditional update: `UPDATE products SET stock = stock - :q WHERE id = :id AND stock >= :q`, then check the affected-row count. If it's 0, return the 409 from the diagram.
  - `SELECT … FOR UPDATE` to lock the row.
  - optimistic locking with a `version` column.
- **Price snapshot.** Copy `unit_price` into `order_items` when the order is placed. Otherwise, a farmer changing a price (story 16) silently rewrites old orders.
- **Money and units.** Use `NUMERIC` / `BigDecimal` for prices, never floats ($0.1 + 0.2 \neq 0.3$). Also decide whether stock is in kg, pieces or bunches, and store the unit explicitly.
- **"Remember me"** (stories 3, 19, 23). A monolith naturally suits **server-side sessions** stored in the DB or Redis, delivered through an `HttpOnly`, `Secure`, `SameSite` cookie. Use a long-lived, *hashed* remember-me token that is rotated on each use. If you pick JWTs instead, you hit the revocation problem: when an admin deletes a malicious user, their JWT stays valid until it expires. Cookies also bring CSRF into play, which `SameSite` plus CSRF tokens handle.
- **Pickup dates.** Store them as `DATE`, not as a timestamp, or timezone conversion will shift them by a day. Validate them server-side: not in the past, and on a day the farm is open, which means you need an opening-hours model.
- **No-shows.** Stock that is reserved but never collected stays locked forever unless a scheduled job releases it. In a monolith that's just a cron-style task inside the app, which is easy, but you have to remember it.
- **Google Maps** (story 13) is the easiest story. Store `lat` / `lng` for each farm and build the link `https://www.google.com/maps/search/?api=1&query=LAT,LNG`. You don't need an API key. The painful part is getting the coordinates in the first place: geocode the farm's address when it registers.
- **Hot products** (story 6) is an aggregate query, for example "most ordered in the last 14 days, in season". Index it properly or cache the result for a few minutes, so it doesn't run a full scan on every homepage load.

Verdict: for NFFH as specified, this is the honest default. Most of its "painful" problems are solved by a database transaction or a single SQL statement, and that stops being true once we split the system up in Architecture B.

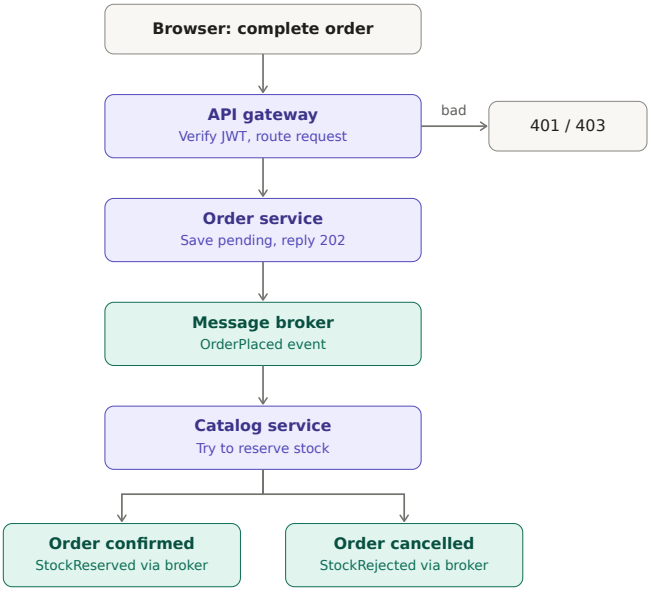
Next up is Architecture B, where the one-transaction checkout from this flowchart becomes the central headache.

📝 notes.txt: added an Architecture A deep-dive section (the flowchart steps, structure, benefits, trade-offs, difficult parts, low-level pain points and verdict). Also added two new open issues: the no-show policy and farm opening hours.

STUDENT

lets continue with arch B

CLAUDE



Flowchart: "Complete order" as a choreographed saga in Architecture B (microservices)

This is the same "complete order" request as before. Compare it with Architecture A's flowchart. There, one database transaction answered "is there stock?" in milliseconds. Here, the order service doesn't own the stock, so it can't check directly. It has to *ask by message* and live with the answer arriving later. Purple boxes are services and teal boxes are events travelling through the broker. Click any box to dig in.

Architecture B: Microservices with an API Gateway

How it's organised

Each service is a separate application with its **own database**. No other service is allowed to touch that database.

Service	Owns	User stories
Identity	users, credentials, roles, sessions/tokens	1–5, 17–25, 27–29
Catalog / Inventory	products, stock, prices, "hot"	6, 8, 14–16, 33–38
Orders	carts, orders, order items, pickup dates	9–12, 26, 40, 41
Geo / Areas	areas, farm locations, map links	7, 13, 30–32, 35, 39

The **API gateway** is the single entry point. It verifies the JWT once, then routes each request to the right service, for example `/api/orders/*` → Orders.

Services talk in two ways:

- **Synchronously (REST/gRPC)** when they need an answer *now*. For example, Orders can fetch product details to show the cart.
- **Asynchronously (events on a broker such as RabbitMQ or Kafka)** to announce that something happened. Examples are `OrderPlaced`, `StockReserved` and `UserDeleted`.

Benefits

- **Independent deployment.** You can ship a new catalog feature without touching Orders.
- **Independent scaling.** Browsing (Catalog) is read-heavy, so you can run 5 copies of it and 1 of Identity.
- **Fault isolation.** If Geo goes down, people can still check out; only the "farms near me" map breaks.
- **Technology freedom.** Each service can pick its own stack. For example, Geo could use PostGIS for distance queries.
- **Team autonomy.** Each team owns a service end to end. This is the real reason companies choose microservices.

Trade-offs

- **No more shared transactions.** Atomic "order + stock" becomes a multi-step *saga*, and the system is **eventually consistent**.
- **Operational cost.** For a minimum setup, you run about 4 services, 4 databases, a broker and a gateway, and you need monitoring for all of them. Local development means a large Docker Compose file.
- **The network is now part of your logic.** Calls can be slow, fail or time out, so every call needs retries, timeouts and fallbacks.
- **Harder debugging.** A single request spans several logs, so you need correlation IDs and distributed tracing (OpenTelemetry).
- **Honestly, overkill for NFFH.** It's a small regional platform, probably built by a small team. You'd pay all the costs without needing the benefits.

Difficult parts (design-level)

1. **Drawing the boundaries.** Where does the "Agricultural Company" live?
    - In **Identity**? It's a user who logs in.
    - In **Geo**? It has a location and an area.
    - In **Catalog**? It's the seller of products.

It's all three, so each service keeps *its own view* of the farm, linked by a shared `farmId`. Getting boundaries wrong produces "chatty" services that constantly call each other. That's a distributed monolith, which is worse than either architecture.

2. **Checkout as a saga.** The flowchart shows a *choreographed* saga: services react to each other's events. The alternative is *orchestration*, where the Order service drives each step explicitly. For every step you need a **compensating action**. If stock is reserved for farm 1 but rejected for farm 2, do you release farm 1's stock or confirm a partial order?
3. **Cross-service queries for admins.** Stories 26–28, 31 and 33 need questions like "all orders, with user names and farm areas". No single database can answer that anymore. The options are:
   - API composition: the gateway calls 3 services and joins the results in memory. This is slow and fragile.
   - A **reporting read model**: a separate store fed by events from all services. This is the CQRS idea.
4. **Deleting a malicious user** (story 25) becomes a `UserDeleted` event. Orders cancels their open orders, and Catalog hides their products if they were a farmer. Each service handles it at its own pace.

Painful low-level details

- **The dual-write problem.** The Order service must save the order *and* publish `OrderPlaced`. If it crashes between the two steps, you either have an order nobody hears about or an event for an order that doesn't exist. The fix is the **transactional outbox**: write the event into an `outbox` table *in the same DB transaction* as the order, and have a relay process publish it afterwards.
- **At-least-once delivery means duplicates.** Brokers will redeliver messages, so the Catalog service may receive `OrderPlaced` twice and reserve stock twice. Consumers must be **idempotent**, typically by keeping a `processed_messages(message_id)` table and skipping messages they've already seen.
- **Ordering.** `ProductUpdated(price=3)` can arrive *after* `ProductUpdated(price=4)` even though it was sent first. You need version numbers on events, or partitioning by `productId` (Kafka).
- **Pending state is visible to the user.** Checkout returns `202 Accepted` with status `PENDING`, so the UI must poll or use WebSockets to show "confirmed" or "sorry, sold out". The user experience is worse than A's instant answer.
- **Duplicated data.** Orders must keep its *own copy* of product name and unit price, plus the farm name and pickup address, at the moment of ordering. In A you'd do that for the price anyway; here you have to do it for everything.
- **JWT revocation, for real this time.** The gateway checks JWTs without calling Identity; that's the whole point of using them. So a banned user stays valid until the token expires. The fixes are short-lived access tokens (about 5–15 min) plus refresh tokens, and a revocation denylist (for example in Redis) that the gateway consults.
- **Service-to-service trust.** Can Orders trust a call claiming to come from Catalog? You need internal auth: either forwarding the user's JWT, or mTLS / service tokens.
- **Schema evolution of events.** Renaming a field in `OrderPlaced` breaks every consumer. Events need versioning and backwards-compatible changes, ideally checked by contract tests (e.g. Pact).

- **Referential integrity is gone.** Geo can delete an area (story 39) while farms in Identity still point to it. No foreign key can stop this across databases. Instead you need a check-before-delete API call or an `AreaDeleted` event with a defined reaction.
- **The "last kilo" race still exists.** It just moves *inside* Catalog, which solves it with the same conditional `UPDATE … WHERE stock >= :q` as in A. Microservices don't remove the problem; they add distributed problems on top of it.

Verdict: this is excellent for learning distributed-systems patterns: sagas, the outbox, idempotency and eventual consistency. It's a poor fit for NFFH's actual size. A useful exam-style point is that a modular monolith (A) with clean boundaries is exactly what you would *later* split into these services if the platform grew.

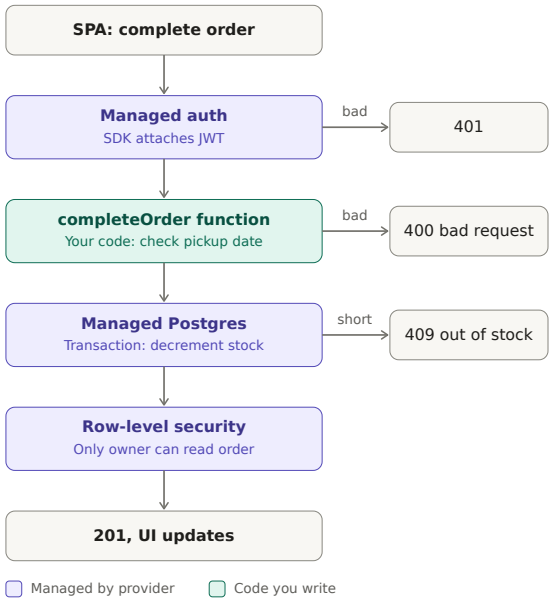
Architecture C comes next. It's interesting because it takes a third path: it keeps one database like A, but removes the server you would otherwise write.

📝 notes.txt: added an Architecture B deep-dive section (the saga flowchart steps, service and story mapping, benefits, trade-offs, difficult parts, low-level pain points and verdict). Also added three open issues: partial failure in the multi-farm saga, where the farm entity lives, and choreography vs orchestration.

STUDENT

lets go with C now, ill have some questions about the architectures after to clear some things up

CLAUDE



Flowchart: "Complete order" in Architecture C (serverless / BaaS)

The flowchart shows the same "complete order" request once more. Look at the colours: almost everything is purple, which means **rented, not written**. The only code you own on this path is the teal function. In A you wrote every layer; here you write one small piece. Like

A, there's one Postgres database, so the stock check is a real transaction again, with none of B's saga.

There's one thing the flowchart can't show: **most requests skip the function entirely.** Browsing products, viewing the cart and a farmer editing their inventory all go *straight from the browser to the database*. Row-level security is the only guard on that path.

Architecture C: Serverless / Backend-as-a-Service

How it's organised

I'll use **Supabase** (managed Postgres + auth + storage + edge functions) as the concrete example. It keeps SQL and transactions, so the comparison with A stays fair. Firebase is the other big option, and I'll note where it differs.

What	How it's built	User stories
Register / login / remember me / logout	Built-in auth, the SDK handles tokens	1–4, 17–20, 22–24 (nearly free)
Simple CRUD	Direct SDK calls to tables, guarded by RLS	5, 8–11, 14–16, 21, 30, 33–39, 40–41
Trusted logic	Edge functions or Postgres functions (RPC)	12 (complete order), 25 (delete user), 29 (modify user)
Computed data	SQL views / materialised views	6 (hot products), 26–27, 31 (admin stats)
Maps	Stored `lat`/`lng` plus a URL, same as A	13

The "backend" you actually write is a set of **SQL migrations** (tables, RLS policies, views, triggers) plus **a handful of functions**.

Benefits

- **Fastest time to working product.** Auth, sessions, password reset, email verification and token refresh are done for you, and that's a third of the user stories.
- **No servers to run.** There's no OS patching or scaling configuration, and the free/cheap tiers comfortably fit NFFH's traffic.
- **Same transactional safety as A.** With Postgres, `completeOrder` is one transaction, so you keep atomic stock decrements. Firestore offers transactions too, but with tighter limits.
- **Built-in extras.** You get realtime subscriptions (a farmer's order list can update live), file storage for product photos, and scheduled jobs (`pg_cron`) for releasing no-show reservations.
- **Small team friendly.** Effort goes into the frontend and the domain rules, not plumbing.

Trade-offs

- **Vendor lock-in.** Your auth, policies and function runtime are provider-specific. Supabase reduces this, since it's open source and you can self-host it. Firebase is much stickier.
- **Logic gets scattered** across the frontend, RLS policies, triggers and functions. There's no single "service layer" where you can read every business rule.

- **Less control.** You live with the platform's limits: function timeouts, cold starts, connection limits and its pricing model.
- **Security by configuration.** Authorisation stops being code you call. It becomes rules the database enforces, and a missing rule fails *open* if you're not careful (details below).

Difficult parts (design-level)

1. **Deciding what may go direct vs. through a function.** The rule of thumb: if an operation touches more than one row in a way that must be consistent, or involves money/stock or another user's data, it goes through a function. Adding to the cart can go direct; completing the order cannot.
2. **Expressing all authorisation as policies.** Ownership checks become database rules, for example:

   ```
   create policy "farmer edits own products" on products
     for update using (seller_id = auth.uid())
     with check (seller_id = auth.uid());
   ```

   Every table needs policies for select/insert/update/delete × client/farmer/admin. That's where most of the real design work goes.
3. **Roles in tokens.** The admin and farmer roles live as *custom claims* inside the JWT. Changing a user's role (story 29) doesn't take effect until their token refreshes. This is the same revocation issue as B, in a smaller form.
4. **Firebase alternative: NoSQL modelling.** With Firestore there are no joins. "Orders with farm names" means **denormalising** (copying data into documents) up front and designing collections around your queries. It's a completely different way of thinking from A.

Painful low-level details

- **RLS fails open if you forget to enable it.** A table created without `enable row level security` is readable and writable by *anyone* with the public anon key, which ships inside your frontend JavaScript. Real-world data leaks happen exactly this way. Make "enable RLS + default deny" part of every migration, and test it.
- **Column-level holes.** RLS protects *rows*, not *columns*. A policy saying "farmer may update own products" also lets them `update products set seller_id = 'someone_else'`, or lets a client update their own order `status = 'COMPLETED'`. You need `with check` clauses, column grants, or triggers that reject changes to protected columns.
- **Never trust the client.** The browser talks to the DB directly, so anyone can open DevTools and send any query the policies allow. Totals, prices and stock must be computed **inside the function/DB**, never sent from the UI.
- **The service role key.** Functions use a secret "service role" key that *bypasses RLS*. If it ever leaks into frontend code or a public repo, all your security is gone.
- **Tokens in the browser.** The SDK stores refresh tokens in `localStorage` by default, so an XSS bug means stolen sessions. That's a weaker default than A's `HttpOnly` cookies, so you need a strict Content-Security-Policy and careful input handling.
- **Deleting a malicious user** (story 25) touches two worlds: the auth user (provider-managed) and your `profiles`, products and orders rows. It needs an admin-only function that does both, plus decisions about their open orders.

- **Cold starts.** The first call to an idle function can take a few hundred ms or more. That's fine for checkout, but noticeable if you put hot reads behind functions.
- **Pricing surprises (mainly Firebase).** Firestore bills **per document read**. A "hot products" homepage reading 50 documents for each visitor costs real money at scale, so you cache or precompute.
- **Environments and drift.** It's tempting to click policies together in the web dashboard. Then dev, staging and prod silently differ. Keep everything as migrations in git and run the local emulator (`supabase start`) for tests.
- **Testing RLS** needs tests that log in *as each role* and attempt forbidden operations. Unit tests of your frontend won't catch a missing policy.
- **Data location (GDPR).** Users and farms are in the EU, so pick an EU region for the project. With some providers, auth or functions may run elsewhere unless configured.

Verdict: this is very likely the *fastest* way to deliver NFFH with a small team, and with Postgres it keeps A's transactional safety. You trade code you control for configuration you must get exactly right. Security becomes a database-policy problem, and you take on vendor dependency.

Ask away. A natural thing to aim for after your questions is a side-by-side comparison of A, B and C against NFFH's requirements.

📝 notes.txt: added an Architecture C deep-dive section (the flowchart steps and colour legend, the direct-to-DB insight, story mapping, benefits, trade-offs, difficult parts, low-level pain points and verdict). Also added two open issues: Supabase vs Firebase, and which operations go direct vs through functions. The next step is now your questions, then the comparison.

STUDENT

i think i currently have a rough understanding of each of the 3 architectures now.

create a table with 3 columns where you will compare each option

1. benefits of implementation
2. benefits of running with the architecture
3. drawbacks of implementation
4. drawbacks of running with the architecture

you can keep the points brief as i can refer back to the longer definitions in the previous messages

CLAUDE

	A. Modular monolith	B. Microservices	C. Serverless / BaaS
1. Implementation benefits	Simplest model to code; one ACID transaction for order + stock; easy refactoring across modules; easy integration tests; one stack to learn	Each service is small and focused; teams build independently; free tech choice per service (e.g. PostGIS); boundaries enforced physically	Least code to write; auth, sessions and token refresh come for free; one Postgres transaction like A; storage, realtime and cron built in; quickest to a working product
2. Running benefits	One artifact plus one DB to deploy; cheap hosting; one stack trace for debugging; easy local dev; plenty for NFFH's traffic	Scale hot parts independently (Catalog); fault isolation (Geo down ≠ checkout down); deploy one service without touching others	No servers to patch or scale; cheap or free tiers fit NFFH; provider handles uptime and backups; scales automatically
3. Implementation drawbacks	Module boundaries kept only by discipline; you write auth, sessions and CSRF yourself; risk of becoming a "big ball of mud"	Sagas and compensating actions; transactional outbox; idempotent consumers; event versioning and contract tests; data duplicated across services; admin queries need composition or a read model; heavy local setup	Authorisation is all RLS policies, which fail open if forgotten; RLS doesn't protect columns; logic scattered across frontend, policies, triggers and functions; harder RLS testing; vendor-specific APIs
4. Running drawbacks	Scales only as a whole; shared fate (one bug can crash everything); every change is a full redeploy	Many moving parts (≈4 services, 4 DBs, broker, gateway); needs tracing and monitoring; eventual consistency (PENDING orders); harder JWT revocation; network failures; overkill for NFFH	Vendor lock-in; platform limits and cold starts; pricing surprises (per-read on Firebase); refresh tokens in localStorage (XSS risk); config drift between environments; must pick an EU region for GDPR

In one line: A is the safe default to build and run. B pays a lot to get scaling and autonomy that NFFH doesn't need yet. C is the fastest to build, but you trade code you control for configuration you must get exactly right.

📝 notes.txt: added section 6 with the full A/B/C comparison across the four criteria and the one-line summary. I also updated the open issues to show the table is done and your clarifying questions are still pending.

STUDENT

if our scope was for example:

- Small region, a few hundred clients, dozens of farms
- Student team of 3, ~3 months, low hosting budget
- Web app only

the most logical architecture to choose in my opinion would be A, no? if so, correct me if my assumption is inaccurate

CLAUDE

Mostly yes. For that scope, A is a correct and easily defensible choice. The one correction is that it's **not the only logical answer**: C is a serious competitor, and on some of your constraints it actually beats A. B is the only option that's clearly wrong.

Why B is out. A few hundred clients and dozens of farms is tiny traffic, so B's main benefit, independent scaling, is worth nothing to you. Its costs land hardest on exactly your constraints: three people would each be running services, sagas, a broker and tracing for three months, and hosting four services plus four databases and a broker doesn't fit a low budget.

Where your assumption holds (A wins):

- **The course goal.** You're graded on *understanding architecture*. A makes the design visible: modules, layers, service interfaces, transactions and testable business rules. In C, much of that design disappears into provider configuration and RLS policies.
- **Control and testability.** All rules live in one codebase you can unit-test and integration-test, and you can review them in one place.
- **Security defaults.** `HttpOnly` session cookies are safer than C's `localStorage` tokens, and authorisation is code you can see, not policies that fail open.
- **No lock-in.** It's also cheap: one small server or a free/low-cost app host plus one Postgres.
- **Future-proof.** A clean modular monolith is exactly what you'd later split into B if NFFH ever grew.

Where C would beat A on your constraints:

- **Time.** In three months, C hands you about a third of the user stories (all the auth ones) for free, which frees time for the farmer and admin features.
- **Budget and ops.** Free tiers cover your scale, and nobody on the team has to be "the server person".
- **Team skills.** If your team is strong in frontend but weak in backend, C lowers the risk of not finishing.
- **One caveat for C.** Free tiers can have limits that matter for a student project. For example, Supabase pauses inactive free projects, which could bite the week before a demo. Check this before choosing.

What would flip the decision:

- Choose **A** if at least one teammate knows a backend framework (Spring, Django, NestJS…) and your grade values design and testing. That's probably your case.
- Choose **C** if your priority is shipping a polished, feature-complete app, the team is frontend-heavy, and you're comfortable writing and testing RLS policies.

You could also take a middle path: build A, but **rent the boring parts**. Use a managed Postgres, and a well-tested auth library from your framework rather than hand-rolled auth. You keep A's visible architecture while cutting some of the work.

So the verdict is **A, with C as a strong runner-up and B ruled out.** In your report, the strongest argument isn't "A is best". It's "A fits *our* constraints, and here's what would make us choose C instead". Showing the trade-off reasoning is what examiners look for.

📝 notes.txt: added section 7 on choosing for your scope (why B is out, why A wins, where C beats A, what would flip the decision, the verdict and a report tip). I also updated the open issues with the tentative choice of A and two things still to confirm: your stack, and whether to rent the boring parts.

STUDENT

Re-attached: NFFH.txt, notes.txt

In A, explain the conditional UPDATE ... WHERE stock >= :q again. Why is reading the stock first and then updating it in code not safe?

CLAUDE

The problem is the **gap between reading the value and writing it back**. By the time you write, the value you checked may already be out of date. This bug is called *check-then-act*, or a *lost update*.

The unsafe version

```
-- step 1: read
SELECT stock FROM products WHERE id = 7;      -- returns 2
-- step 2: decide in code
if (stock >= quantity) {
    -- step 3: write the value computed in code
    UPDATE products SET stock = 0 WHERE id = 7;   -- 2 - 2 = 0
}
```

Each step is correct on its own, but two checkouts can run at the same time. Say there are 2 kg of tomatoes left, and Anna and Ben each order 2 kg at the same moment:

Time	Anna's request	Ben's request	Stock in DB
t1	reads stock = 2		2
t2		reads stock = 2	2
t3	checks 2 ≥ 2 ✓		2
t4		checks 2 ≥ 2 ✓	2
t5	writes stock = 0		0
t6		writes stock = 0	0

Both get "order confirmed". You've sold 4 kg and only 2 kg exist. Ben's write simply overwrote Anna's, so the database shows 0 and hides the fact that it went wrong. If your code wrote `stock = stock - 2` instead, you'd see −2, which is at least visible.

Why this happens even in a monolith. One app still handles many requests in parallel on different threads, and you may run two copies of it behind a load balancer. Concurrency is the normal state for any web server.

Two "fixes" that don't work:

- **Wrapping it in `BEGIN … COMMIT`.** Postgres' default isolation level, *READ COMMITTED*, still lets both transactions read 2 before either commits. A transaction alone doesn't stop this.
- **A lock in your code (`synchronized`, a mutex).** It only works inside one process. It breaks the moment you run a second replica.

The safe version

```
UPDATE products
SET    stock = stock - :q
WHERE  id = :id AND stock >= :q;
```

Then your code checks **how many rows were affected**:

- **1 row:** stock was reserved, so continue with the order.
- **0 rows:** there wasn't enough stock, so return the 409 from the flowchart.

The check (`stock >= :q`) and the change (`stock - :q`) are now **one statement inside the database**, so there's no gap for another request to slip into. Here's the same race replayed:

1. Anna's UPDATE runs first and locks that row.
2. Ben's UPDATE arrives and **waits** for the lock.
3. Anna commits, so stock = 0.
4. Postgres **re-checks Ben's WHERE against the new value**: 0 ≥ 2 is false, so 0 rows are updated and Ben gets "sorry, sold out".

You can also add `CHECK (stock >= 0)` to the table as a safety net. If any future code path forgets the rule, the database refuses the write.

Alternatives (same idea: make check + write indivisible)

- **Pessimistic locking (`SELECT … FOR UPDATE`).** This locks the row when you read it, so others wait until you commit. Use it when your rule is too complex for one `WHERE`. Keep the transaction short, and in multi-item carts lock rows in a fixed order (e.g. by product id) to avoid deadlocks.
- **Optimistic locking (a `version` column).** The update is `UPDATE … WHERE id = :id AND version = :v`. If it affects 0 rows, someone else changed the row first, so you retry. This is good when conflicts are rare, which is likely for NFFH.
- **`SERIALIZABLE` isolation.** The database detects the conflict and aborts one transaction, and your code must retry it.

For a cart with several products: run the conditional UPDATE for each item **inside one transaction**. If any item returns 0 rows, `ROLLBACK` the whole order, so you never end up with half an order reserved.

How to test it: a normal unit test will never catch this. Write an integration test that fires 10 parallel checkouts at the last 2 kg using a real database. Then assert that exactly one succeeds and stock never goes below 0.

File sent: notes.txt

📝 notes.txt: added section 8, "Conditional UPDATE vs read-then-write", with the Anna/Ben timeline, why transactions and in-code locks don't help, how the atomic UPDATE behaves, the alternatives, multi-item carts, and testing. The notes file is now in your working folder because the original copy had become read-only.

STUDENT

is option C more prone to things like data leaks by mistake? from a first glance it seems so, what could by example happen

CLAUDE

Yes, your first impression is right. C is more prone to **accidental** leaks. That's not because Supabase or Firebase are insecure; it's because the defaults are reversed:

- **In A, access is closed by default.** Data is only reachable if someone writes an endpoint for it. The endpoint also decides which fields go out, through the DTO (the object the API serialises). Forget something and the data stays hidden.
- **In C, access is open by default.** Every table is already an API that anyone can call from the browser using the public anon key, which ships in your JavaScript. What protects the data is the rules you remembered to write. Forget something and the data is exposed.

So in C, every mistake of omission becomes a leak. Here are realistic ways it could happen in NFFH:

1. A new table without RLS

Halfway through the project, someone adds a table such as `order_notes` ("I'll arrive around 5 pm, call me on +421...") and forgets `enable row level security`. Anyone can then open DevTools and run `supabase.from('order_notes').select('*')`. That returns **every client's notes and phone numbers**, with no login needed.

2. A policy that's correct for rows but leaks columns

Clients need to see farms, so you write a policy: "authenticated users can read `profiles`." But `profiles` holds farmers *and clients*, with email, phone and address. The UI only shows farm names, but `select('*')` returns **every user's personal data**. The screen looks fine; the network response doesn't. In A, your DTO would have sent only `name` and `location`.

3. A user makes themselves admin (privilege escalation)

You store `role` in the `profiles` table and add the reasonable policy "users can update their own profile" for story 5. A user then runs:

```
supabase.from('profiles').update({ role: 'admin' }).eq('id', myId)
```

The row is their own, so RLS allows it. They now have the admin features: delete users (25), see all orders (26), and see all user details (28). This is the worst one, because one missing column restriction gives away the whole platform.

4. Changing what you shouldn't, on your own rows

"Clients can update their own orders" (to change the pickup date) also lets a client set `status = 'COMPLETED'` or change `quantity` after stock was reserved. "Farmers update own products" lets them change `seller_id`. That isn't a data *leak*, but it's the same kind of mistake: the policy checks *which row*, not *which column*.

5. An admin view that bypasses RLS

For stories 26 and 31, you create a view `admin_order_stats`. In Postgres, views by default run **with the creator's rights, not the caller's**, so RLS on the underlying tables is skipped. If the view is in the exposed schema, anyone can read **all orders with names and pickup dates**. (Supabase's security advisor flags exactly this; the fix is `security_invoker = true`.)

6. An "admin" function that anyone can call

You write a Postgres function `get_user_details(user_id)` for story 28, but forget to check inside it that the caller is an admin. Functions in the exposed schema are callable by anyone via RPC, so any client can **look up any user**.

7. The service role key ends up in the frontend

This key bypasses all RLS. A teammate puts it in `VITE_SUPABASE_KEY` "because the anon one gave a permission error". Any variable with that prefix is bundled into the public JavaScript. Or the key gets committed to a public GitHub repo. Either way, the **entire database is open for reading and writing**. Bots scan GitHub for these keys within minutes.

8. A public storage bucket

Product photos go in a public bucket, which is fine. Later, farmers upload a business registration document to the *same* bucket. Its files are reachable by URL by anyone.

9. Realtime broadcasts

You subscribe the farmer's order screen to changes on `orders`. If the table or channel isn't covered by RLS, **every connected client receives everyone's new orders** live.

10. Firebase's "test mode"

Firestore projects often start with rules like `allow read, write: if request.time < <date>`. Teams forget, ship it, and the database is world-writable until that date. This is one of the most common real-world leak sources.

To be fair to A

A has the same *types* of bugs:

- **IDOR:** a missing ownership check on `GET /orders/42`.
- **Mass assignment:** binding the whole request body to an entity, so `{"role":"admin"}` gets saved. This is the A version of #3.

The difference is how many places you have to get right. In A, you mostly have to secure the endpoints you wrote. In C, you have to secure every table, every column, every view, every function, every bucket and every realtime channel, and the default for anything you forget is "open".

How you'd defend C (if you chose it)

- **Default deny everywhere.** Every migration enables RLS on every table, then adds policies. Run Supabase's security advisor before each release.
- **Keep sensitive data out of the exposed schema.** Put phone and address in a separate table, or a `private` schema, so it's not reachable via the API at all.
- **Never let users write security fields.** Store `role` in the auth user's `app_metadata` (users can't edit it), not in `profiles`. Use column-level `grant update (pickup_date)` or triggers to protect the other columns.
- **Make views and functions safe.** Use `security_invoker = true` on views. Check `is_admin()` inside every privileged function, and `revoke execute … from anon`.
- **Select explicit columns**, never `*`. Treat this as convenience only, not as security.
- **Write tests per role**: log in as client, farmer and anonymous, and attempt every forbidden read and write.
- **Use secret scanning** on the repo, with a team rule that only the anon key ever goes in frontend env vars.

The takeaway for your report: C isn't less secure in theory, but its failure mode is "open". Your security depends on configuration being complete, not on code being correct. A small student team under deadline pressure is exactly where configuration gets forgotten, which is another argument for A in your scope.

File sent: notes.txt

📝 notes.txt: added section 9, "Is C more prone to accidental data leaks?". It covers the closed-vs-open-by-default explanation, the 10 NFFH leak examples (privilege escalation through `role` is the worst one), the equivalent bugs in A (IDOR and mass assignment), how you'd defend C, and the takeaway for your report.

STUDENT

would it make sense to claim that A is the safer option when concerned with security risks when the development team lacks real experience with these architectures?

CLAUDE

Yes, it's a defensible claim, but only if you word it precisely and attach one condition. As written ("A is the safer option"), an examiner could rightly push back, because A is safer in some areas and riskier in others.

Why the claim holds

For an inexperienced team, what matters isn't which architecture is *most secure when done perfectly*. It's **what happens when you make mistakes**, because you will.

- **Failure mode.** In A, a forgotten rule usually means a feature *doesn't work* (there's no endpoint, or a 403). In C, a forgotten rule usually means data is *exposed* (section 9). Mistakes in A tend to be visible bugs; in C they're silent leaks.
- **Attack surface.** In A, the only way in is through the endpoints you wrote. In C, it's every table, column, view, function, bucket and channel.
- **Reviewability.** In A, authorisation is code in one service layer, so a teammate can read it in a pull request. In C, it's spread across SQL policies, grants, triggers and dashboard settings.
- **Testability.** Your existing unit and integration test skills apply directly in A. In C, you need a new kind of test: logging in as each role and attempting forbidden queries.
- **Familiarity.** Students usually know "write an endpoint, check the user, return a DTO". Database-enforced security (RLS, `security_invoker`, column grants) is a new mental model, and new mental models are where mistakes happen.

Where the claim breaks (and the condition)

A is **riskier** than C in one important area: **authentication**. C gives you battle-tested login, password hashing, token refresh, password reset and email verification. An inexperienced team that *hand-writes* these in A can easily make classic mistakes:

- storing passwords with SHA-256 instead of bcrypt/argon2
- predictable or non-expiring reset tokens
- no CSRF protection
- sessions not invalidated on logout
- no rate limiting on login

A also makes you responsible for **running a server**: patching it, keeping secrets out of the repo, using TLS, and not exposing the database port to the internet. C handles all of that for you.

So the claim is only true if the condition is met: **A is safer provided the team uses the framework's established security module** (Spring Security, Django's auth, ASP.NET Identity, Passport/NestJS guards) and a managed host or database instead of hand-rolling them. That's the "rent the boring parts" middle path from section 7.

Where B fits

B is the riskiest option for an inexperienced team. It has everything A has, plus service-to-service authentication, JWT revocation, broker credentials, many more secrets and many more exposed components. It's worth one sentence in your report.

How to phrase it in your report

Avoid: "*Architecture A is more secure.*"

Better: "*Given a team with limited experience, we consider Architecture A the lower-risk option from a security perspective. Its failure mode is closed: a missing authorisation rule results in a denied or missing feature rather than exposed data. Its attack surface is limited to explicitly implemented endpoints, and its authorisation logic is code that can be reviewed and tested with familiar techniques. This holds on the condition that authentication is implemented with the framework's established security module rather than custom code. Architecture C provides stronger authentication out of the box, but its open-by-default data access means that configuration omissions lead directly to data exposure.*"

That version states the claim, gives the reasons, names the condition, and acknowledges where the alternative is stronger. That last part is what turns an opinion into an argument.

File sent: notes.txt

📝 notes.txt: added section 10 on the security claim for an inexperienced team: why it holds, where it breaks (authentication and server operations), the condition, where B fits, and the suggested report wording. I also updated the open issues: using the framework's auth module is now marked as required for this claim, not optional.

Appendix: attached file NFFH.txt

```
# SYSTEM DESCRIPTION:

Not Far(m) From Home is a platform that allows a direct interaction between local farmers and consumers, with the main purpose of being “km 0”.
The Farmers will be able to post their fresh produce in the site, and the consumers to reserve the produce and select a day for the pickup at the Agricoltural Company.

# USER STORIES:

1) As a Client , I want to Register in the site so that I can use the site
2) As a Client , I want to login in the site so that I can use the site
3) As a Client , I want to not put my Credentials in the site every time a reload the site, so that I can use the site
4) As a Client , I want to logout, so that no one else use my account
5) As a Client , I want to see my personal information
6) As a Client , I want to See Hot products, so that i can discover the product in the season
7) As a Client , I want to See Agricultural companies in my area, so that i can choose where to buy products
8) As a Client , I want to See products for each Agricultural company, so that I can buy From them
9) As a Client , I want to Add products to my cart, so that i can buy them
10) As a Client, I want to Remove products from my cart, so that i can decide what to buy
11) As a Client , I want to see the products in my cart, so that i can see want I am going to buy
12) As a Client , I want to Complete an order, so that I can choose a date to go and pick up the products
13) As a Client , I want to Open in google Maps the Location of the Agricultural company, so that i can find directions to it easly
14) As an Agricultural Company, I want to Add products in inventory, so that I can show my clients the new produce
15) As an Agricultural Company, I want to Remove products in inventory, so that My clients don't try to buy an item that I don't have anymore
16) As an Agricultural Company, I want to Modify products in inventory, so that I can change price and quantities on an item
17) As an Agricultural Company, I want to Registrer in the site, so that i can be visible and start doing business in the site
18) As an Agricultural Company, I want to Login, so that i can work on the site
19) As an Agricultural Company, I want to not put my Credentials in the site every time a reload the site, so that I can use the site easily
20) As an Agricultural Company, I want to logout from the site
21) As an Agricultural Company, I want to see my personal information
22) As the Administrator of the site, I want to login in the site
23) As the Administrator of the site, I want to not put my Credentials in the site every time a reload the site
24) As the Administrator of the site, I want to logout from the site
25) As the Administrator of the site, I want to delete malevolus user
```

```
26) As the Administrator of the site, I want to see all orders, so that I can better analyze how the platform is doing
27) As the Administrator of the site, I want to see all users, so that I can better analyze how the platform is doing
28) As the Administrator of the site, I want to get user details, so that I can have some clear view over the user
29) As the Administrator of the site, I want to modify users, so that I can do site maintenance
30) As the Administrator of the site, I want to see all areas, so that I can do site maintenance
31) As the Administrator of the site, I want to get all Agricultural company by areas, so that I can analyze how they are distributed
32) As the Administrator of the site, I want to get all Agricultural company details, so that I can do platform maintenance
33) As the Administrator of the site, I want to get product by seller, so that I can do site maintenance
34) As the Administrator of the site, I want to get product by id, so that I can do site maintenance
35) As the Administrator of the site, I want to add an area, so that if there is some request for that area to be added, I can
36) As the Administrator of the site, I want to add product, so that I can do site maintenance
37) As the Administrator of the site, I want to modify product, so that I can do site maintenance
38) As the Administrator of the site, I want to delete product, so that I can do site maintenance
39) As the Administrator of the site, I want to delete an area, so that I can do site maintenance
40) As a client, i want to see all my orders
41) As a farmer, i want to see all my orders
```